Week 9

Sales and customer analytics system

Scenario:

The e-commerce company aims to study its sales information to understand customer buying habits, product performance, and revenue patterns. The management team requires reports to find the most valuable customers, best-selling products, and profitable product categories.

Required Tables and Their Attributes:

s.no	Table name	Attributes
1	CUSTOMER	CUSTOMER_ID, CUSTOMER_NAME, EMAIL, CITY
2	ORDERS	ORDER_ID, CUSTOMER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS
3	PRODUCT	PRODUCT_ID, PRODUCT_NAME, PRICE, CATEGORY_ID
4	CATEGORY	CATEGORY_ID, CATEGORY_NAME
5	ORDER_ITEMS	ORDER_ITEM_ID, ORDER_ID, PRODUCT_ID, QUANTITY
6	PAYMENT	PAYMENT_ID, ORDER_ID, PAYMENT_DATE, PAYMENT_AMOUNT, PAYMENT_METHOD, PAYMENT_STATUS

1. **Apply SQL Aggregate Functions**

COUNT() – Counts the number of records.

SUM() – Calculates the total value.

AVG() – Finds the average value.

MIN() – Finds the minimum value.

MAX() – Finds the maximum value.

2. **Generate Total Sales Reports**

TASKS:

a. Total number of orders placed.

This query is used to find the total number of orders placed by customers. The COUNT() aggregate function is used to count the number of orders in the Orders table.

```
SQL> SELECT COUNT(*) AS number_of_orders_placed FROM orders;

NUMBER_OF_ORDERS_PLACED
-----------------------
                     10
```

b. Total revenue generated from sales.

This query is used to calculate the total revenue generated from all sales. The `SUM()` aggregate function is used to add all the order amounts in the Orders table.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
  2  FROM ORDERS;

TOTAL_REVENUE
-------------
       280000
```

c. Average order value.

This query is used to calculate the average value of all orders placed by customers. The AVG() aggregate function is used to find the average order amount in the Orders table.

```
SQL> SELECT AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
  2  FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
              28000
```

d. Highest and lowest order amount.

This query is used to find the highest and lowest order amounts from all customer orders. The MAX() function finds the highest order amount, and the MIN() function finds the lowest order amount in the Orders table.

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER,
  2         MIN(TOTAL_AMOUNT) AS LOWEST_ORDER
  3  FROM ORDERS;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
        55000        12000
```

e. Total sales for a specific period.
This query is used to calculate the total sales generated during a specific time period. The SUM() function adds all the order amounts, and the WHERE condition filters orders based on the selected dates.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM ORDERS
  3  WHERE ORDER_DATE BETWEEN
  4  TO_DATE('01-10-2026','DD-MM-YYYY')
  5  AND TO_DATE('31-10-2026','DD-MM-YYYY');

TOTAL_SALES
-----------
     280000
```

3. Customer Purchase Analysis
   a. Total number of orders placed by each customer.
   This query is used to find the total number of orders placed by each customer. The COUNT() function counts the number of orders, and the GROUP BY clause groups the orders based on each customer

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
          1 Arun                                                          1
          9 Meena                                                         1
          2 Priya                                                         1
          3 Kaviya                                                        1
          4 Rahul                                                         1
          7 Sneha                                                         1
         10 Ajay                                                          1
          6 Vijay                                                         1
          8 Karthik                                                       1
          5 Divya                                                         1

10 rows selected.
```

   b. Total amount spent by each customer.
   This query is used to calculate the total amount spent by each customer. The SUM() function adds all the order amounts, and the GROUP BY clause groups the spending based on each customer.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         NVL(SUM(O.TOTAL_AMOUNT), 0) AS AMOUNT_SPENT
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_NAME                                      AMOUNT_SPENT
----------- -------------------------------------------------- ------------
          1 Arun                                                      55000
          9 Meena                                                     35000
          2 Priya                                                     30000
          3 Kaviya                                                    15000
          4 Rahul                                                     25000
          7 Sneha                                                     18000
         10 Ajay                                                      28000
          6 Vijay                                                     12000
          8 Karthik                                                   22000
          5 Divya                                                     40000

10 rows selected.
```

c. Average spending per customer.

This query is used to calculate the average amount spent by each customer. The AVG() function finds the average order amount, and the GROUP BY clause groups the spending based on each customer

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         NVL(AVG(O.TOTAL_AMOUNT), 0) AS AVERAGE_SPENDING
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_NAME                                  AVERAGE_SPENDING
----------- -------------------------------------------------- ---------------
          1 Arun                                                      55000
          9 Meena                                                     35000
          2 Priya                                                     30000
          3 Kaviya                                                    15000
          4 Rahul                                                     25000
          7 Sneha                                                     18000
         10 Ajay                                                      28000
          6 Vijay                                                     12000
          8 Karthik                                                   22000
          5 Divya                                                     40000

10 rows selected.
```

d. Customers with maximum purchase amount.
This query identifies the customer who has spent the highest total amount on purchases. It uses the SUM() function to calculate each customer's total spending and a subquery with MAX() to find the maximum purchase amount.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING SUM(O.TOTAL_AMOUNT) = (
  9      SELECT MAX(TOTAL_SPENT)
 10      FROM (
 11          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENT
 12          FROM ORDERS
 13          GROUP BY CUSTOMER_ID
 14      )
 15  );

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_PURCHASE
----------- -------------------------------------------------- --------------
          1 Arun                                                        55000
```

e. Customers with fewer purchases.
This query identifies customers who have placed fewer orders. It uses the COUNT() function to count the number of orders placed by each customer and sorts the results in ascending order.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  4  FROM CUSTOMER C
  5  LEFT JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING COUNT(O.ORDER_ID) <= 2;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
          1 Arun                                                          1
          9 Meena                                                         1
          2 Priya                                                         1
          3 Kaviya                                                        1
          4 Rahul                                                         1
          7 Sneha                                                         1
         10 Ajay                                                          1
          6 Vijay                                                         1
          8 Karthik                                                       1
          5 Divya                                                         1

10 rows selected.
```

4. Find Top Customers Based on Purchase Amount
   a. Top 5 customers based on total spending.
      This query identifies the top 5 customers who have spent the highest amount on purchases. It uses the SUM() function to calculate total spending and ORDER BY DESC to arrange customers from highest to lowest spending.

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.CUSTOMER_NAME,
  5             SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING
  6      FROM CUSTOMER C
  7      JOIN ORDERS O
  8      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9      GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10      ORDER BY SUM(O.TOTAL_AMOUNT) DESC
 11  )
 12  WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENDING
----------- -------------------------------------------------- --------------
          1 Arun                                                        55000
          5 Divya                                                       40000
          9 Meena                                                       35000
          2 Priya                                                       30000
         10 Ajay                                                        28000
```

   b. Customers with maximum number of orders.
      This query identifies the customer who has placed the highest number of orders. It uses the COUNT() function to count orders and MAX() to find the maximum number of orders.

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.CUSTOMER_NAME,
  5             COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  6      FROM CUSTOMER C
  7      JOIN ORDERS O
  8      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9      GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10      ORDER BY COUNT(O.ORDER_ID) DESC
 11  )
 12  WHERE ROWNUM = 1;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
          1 Arun                                                          1
```

c. Frequent customers.

This query identifies frequent customers who place orders regularly. It uses the COUNT() function to count the number of orders placed by each customer and displays customers with 3 or more orders.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         COUNT(O.ORDER_ID) AS TOTAL_ORDERS
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING COUNT(O.ORDER_ID) >= 3
  9  ORDER BY TOTAL_ORDERS DESC;

no rows selected
```

d. High-value customers.

This query identifies high-value customers who have spent more than ₹50,000 on purchases. It uses the SUM() function to calculate each customer's total spending and the HAVING clause to filter customers based on their purchase amount.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING SUM(O.TOTAL_AMOUNT) >= 50000
  9  ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENDING
----------- -------------------------------------------------- --------------
          1 Arun                                                        55000
```

5. Identify Best-Selling Products

a. Products with maximum sales quantity.

This query identifies the product that has been sold in the highest quantity. It uses the SUM() function to calculate the total quantity sold for each product and MAX() to find the maximum sales quantity.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OI.QUANTITY) AS TOTAL_QUANTITY
  4  FROM PRODUCT P
  5  JOIN ORDER_ITEMS OI
  6  ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY SUM(OI.QUANTITY) DESC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_QUANTITY
---------- -------------------------------------------------- --------------
       205 Mouse                                                           4
       203 Headphone                                                       3
       208 Smartwatch                                                      3
       202 Mobile                                                          2
       210 Speaker                                                         2
       204 Keyboard                                                        2
       207 Monitor                                                         2
       201 Laptop                                                          1
       206 Tablet                                                          1
       209 Printer                                                         1

10 rows selected.
```

b. Products generating highest revenue.
   This query identifies the product that generates the highest revenue. It uses the SUM() function to calculate the total revenue for each product by multiplying the product price by the quantity sold. It uses MAX() to find the highest revenue.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(P.PRICE * OI.QUANTITY) AS TOTAL_REVENUE
  4  FROM PRODUCT P
  5  JOIN ORDER_ITEMS OI
  6  ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_REVENUE DESC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_REVENUE
---------- -------------------------------------------------- -------------
       201 Laptop                                                     55000
       202 Mobile                                                     30000
       206 Tablet                                                     25000
       207 Monitor                                                    24000
       208 Smartwatch                                                 15000
       209 Printer                                                    10000
       203 Headphone                                                   6000
       210 Speaker                                                     6000
       205 Mouse                                                       3200
       204 Keyboard                                                    3000

10 rows selected.
```

c. Least-selling products.
   This query identifies the product that has been sold in the lowest quantity. It uses the SUM() function to calculate the total quantity sold for each product and MIN() to find the minimum sales quantity.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         SUM(OI.QUANTITY) AS TOTAL_QUANTITY
  4  FROM PRODUCT P
  5  JOIN ORDER_ITEMS OI
  6  ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  ORDER BY TOTAL_QUANTITY ASC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_QUANTITY
---------- -------------------------------------------------- --------------
       201 Laptop                                                          1
       206 Tablet                                                          1
       209 Printer                                                         1
       210 Speaker                                                         2
       202 Mobile                                                          2
       207 Monitor                                                         2
       204 Keyboard                                                        2
       203 Headphone                                                       3
       208 Smartwatch                                                      3
       205 Mouse                                                           4

10 rows selected.
```

Products requiring promotion.

This query identifies products with low sales quantities that may require promotion to increase their sales. It uses the SUM() function to calculate the total quantity sold and displays products with fewer than 5 units sold.

```
SQL> SELECT P.PRODUCT_ID,
  2         P.PRODUCT_NAME,
  3         NVL(SUM(OI.QUANTITY), 0) AS TOTAL_QUANTITY
  4  FROM PRODUCT P
  5  LEFT JOIN ORDER_ITEMS OI
  6  ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  8  HAVING NVL(SUM(OI.QUANTITY), 0) <= 2
  9  ORDER BY TOTAL_QUANTITY ASC;

PRODUCT_ID PRODUCT_NAME                                       TOTAL_QUANTITY
---------- -------------------------------------------------- --------------
       201 Laptop                                                          1
       209 Printer                                                         1
       206 Tablet                                                          1
       202 Mobile                                                          2
       207 Monitor                                                         2
       210 Speaker                                                         2
       204 Keyboard                                                        2

7 rows selected.
```

6. Perform Category-wise Sales Analysis
   a. Total sales generated by each category.
      This query calculates the total sales generated by each product category. It uses the SUM() function to calculate total sales and GROUP BY to display the results category-wise.

```
SQL> SELECT C.CATEGORY_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SALES
  2  FROM CATEGORY C
  3  JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID
  4  JOIN ORDER_ITEMS OI ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  JOIN ORDERS O ON OI.ORDER_ID = O.ORDER_ID
  6  GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                                      TOTAL_SALES
-------------------------------------------------- -----------
Clothing                                                 42000
Books                                                    65000
Accessories                                              35000
Electronics                                              55000
Furniture                                                83000
```

   b. Number of products sold in each category.
      This query calculates the total number of products sold in each category. It uses the SUM() function to add the product quantities and GROUP BY to display the results category-wise.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OI.QUANTITY) AS TOTAL_PRODUCTS_SOLD
  2  FROM CATEGORY C
  3  JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID
  4  JOIN ORDER_ITEMS OI ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                                  TOTAL_PRODUCTS_SOLD
---------------------------------------------- -------------------
Clothing                                                         3
Books                                                            8
Accessories                                                      1
Electronics                                                      1
Furniture                                                        8
```

   c. Highest revenue-generating category.

      This query identifies the category that generates the highest revenue. It uses the SUM() function to calculate total revenue for each category and MAX() to find the highest revenue-generating category.

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CATEGORY_NAME,
  4             SUM(P.PRICE * OI.QUANTITY) AS TOTAL_REVENUE
  5      FROM CATEGORY C
  6      JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID
  7      JOIN ORDER_ITEMS OI ON P.PRODUCT_ID = OI.PRODUCT_ID
  8      GROUP BY C.CATEGORY_NAME
  9      ORDER BY TOTAL_REVENUE DESC
 10  )
 11  WHERE ROWNUM = 1;

CATEGORY_NAME                                      TOTAL_REVENUE
-------------------------------------------------- -------------
Clothing                                                   55000
```

d. Average sales per category.

This query calculates the average sales amount for each product category. It uses the AVG() function to find the average sales and GROUP BY to display the results category-wise.

```
SQL> SELECT C.CATEGORY_NAME, AVG(O.TOTAL_AMOUNT) AS AVERAGE_SALES
  2  FROM CATEGORY C
  3  JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID
  4  JOIN ORDER_ITEMS OI ON P.PRODUCT_ID = OI.PRODUCT_ID
  5  JOIN ORDERS O ON OI.ORDER_ID = O.ORDER_ID
  6  GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                                      AVERAGE_SALES
-------------------------------------------------- -------------
Clothing                                                   21000
Books                                                  21666.6667
Accessories                                                35000
Electronics                                                55000
Furniture                                              27666.6667
```

7. Generate Business Analytics Reports

Report 1: Sales Performance Report
- Total sales
- Number of orders
- Average order value Highest order value.

This report analyzes the overall sales performance of the business. It displays total sales, number of orders, average order value, and highest order value using SQL aggregate functions

```
SQL> SELECT
  2      SUM(TOTAL_AMOUNT) AS TOTAL_SALES,
  3      COUNT(ORDER_ID) AS NUMBER_OF_ORDERS,
  4      AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE,
  5      MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER_VALUE
  6  FROM ORDERS;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- --------------- ------------------ ------------------
     280000              10              28000              55000
```

Report 2: Customer Analytics Report

- Customer name
- Number of orders
- Total spending
- Purchase frequency.

This report analyzes customer purchasing behaviour. It displays customer name, number of orders, total spending, and purchase frequency using SQL aggregate function

```
SQL> SELECT C.CUSTOMER_NAME,
  2         COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING,
  4         COUNT(O.ORDER_ID) AS PURCHASE_FREQUENCY
  5  FROM CUSTOMER C
  6  JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_NAME;

CUSTOMER_NAME                                      NUMBER_OF_ORDERS
-------------------------------------------------- ----------------
TOTAL_SPENDING PURCHASE_FREQUENCY
-------------- ------------------
Karthik                                                           1
         22000                  1

Meena                                                             1
         35000                  1

Sneha                                                             1
         18000                  1


CUSTOMER_NAME                                      NUMBER_OF_ORDERS
-------------------------------------------------- ----------------
TOTAL_SPENDING PURCHASE_FREQUENCY
-------------- ------------------
Ajay                                                              1
         28000                  1

Priya                                                             1
         30000                  1

Rahul                                                             1
         25000                  1
```

```
CUSTOMER_NAME                                      NUMBER_OF_ORDERS
-------------------------------------------------- ----------------
TOTAL_SPENDING PURCHASE_FREQUENCY
-------------- ------------------
Divya                                                             1
         40000                  1

Kaviya                                                            1
         15000                  1

Vijay                                                             1
         12000                  1


CUSTOMER_NAME                                      NUMBER_OF_ORDERS
-------------------------------------------------- ----------------
TOTAL_SPENDING PURCHASE_FREQUENCY
-------------- ------------------
Arun                                                              1
         55000                  1


10 rows selected.
```

Report 3: Product Performance Report

- Product name
- Quantity sold
- Revenue generated

This report analyzes the performance of each product. It displays the product name, quantity sold, and revenue generated using SQL aggregate functions.

```
SQL> SELECT P.PRODUCT_NAME,
  2         SUM(OI.QUANTITY) AS QUANTITY_SOLD,
  3         SUM(P.PRICE * OI.QUANTITY) AS REVENUE_GENERATED
  4  FROM PRODUCT P
  5  JOIN ORDER_ITEMS OI ON P.PRODUCT_ID = OI.PRODUCT_ID
  6  GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
Mobile                                                         2
            30000

Tablet                                                         1
            25000

Printer                                                        1
            10000


PRODUCT_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
Laptop                                                         1
            55000

Mouse                                                          4
             3200

Speaker                                                        2
             6000


PRODUCT_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
Headphone                                                      3
             6000

Keyboard                                                       2
             3000

Smartwatch                                                     3
            15000


PRODUCT_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
Monitor                                                        2
            24000


10 rows selected.
```

Report 4: Category Analysis Report

- Category name
- Total products sold
- Total revenue

This report analyzes the sales performance of each product category. It displays the category name, total products sold, and total revenue generated using SQL aggregate functions.

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OI.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
  3         SUM(P.PRICE * OI.QUANTITY) AS TOTAL_REVENUE
  4  FROM CATEGORY C
  5  JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID
  6  JOIN ORDER_ITEMS OI ON P.PRODUCT_ID = OI.PRODUCT_ID
  7  GROUP BY C.CATEGORY_NAME;

CATEGORY_NAME                                      TOTAL_PRODUCTS_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Clothing                                                             3
        55000

Books                                                                8
        27000

Accessories                                                          1
        10000


CATEGORY_NAME                                      TOTAL_PRODUCTS_SOLD
-------------------------------------------------- -------------------
TOTAL_REVENUE
-------------
Electronics                                                          1
        55000

Furniture                                                            8
        30200
```

Conclusion

Thus, the Sales and Customer Analytics System helps analyze sales data, customer purchasing behavior, product performance, and category-wise revenue using SQL queries and aggregate functions. It helps identify top customers, best-selling products, and revenue trends, supporting better business decisions.